

Entrega Final — MedCare-CLI

Aluno responsável: Nelson Ribeiro Felix

Matrícula: preencher antes do envio

Projeto: MedCare-CLI

Repositório público: <https://github.com/NelsonRibeiroFelix/medcare-cli>

Aplicação publicada: [Executar Online via Google Colab](#)

Descrição do Projeto

O **MedCare-CLI** é uma aplicação de linha de comando em Python criada para auxiliar no controle de medicamentos. O sistema permite cadastrar, listar e remover medicamentos com nome, dosagem e horário de uso. A versão final foi preparada para persistir dados em um banco Supabase, serviço baseado em PostgreSQL hospedado em nuvem, atendendo ao requisito de leitura e escrita em banco remoto.¹

Tecnologias e Evidências de Entrega

Critério	Atendimento no projeto
Banco de dados em nuvem	Integração configurável com Supabase no arquivo <code>src/database_service.py</code> e schema em <code>supabase/schema.sql</code> .
Testes automatizados	Suíte com <code>pytest</code> , incluindo testes de regras de medicamento e integração com ViaCEP simulada por mock.
Integração contínua	Workflow em <code>.github/workflows/ci.yml</code> executando testes e lint em pushes e Pull Requests para <code>main</code> .
Deploy funcional	Execução demonstrativa via Google Colab vinculada ao repositório público.
Documentação	README atualizado com stack, instruções de Supabase, comandos de execução, testes e autoria.
Autoria	Arquivos de metadados e documentação identificam Nelson Ribeiro Felix como autor do projeto.

Banco de Dados

A tabela `medications` deve ser criada no Supabase com o script `supabase/schema.sql`. Depois da criação da tabela, a aplicação deve receber as variáveis `SUPABASE_URL` e `SUPABASE_ANON_KEY`, conforme o modelo `.env.example`. A ausência dessas variáveis ativa apenas o fallback local de desenvolvimento, portanto a execução final avaliada deve usar as credenciais reais do projeto Supabase.

Validação de Qualidade

A qualidade do projeto foi validada localmente com os comandos abaixo. O workflow do GitHub Actions usa a mesma lógica para automatizar a validação em Pull Requests, prática recomendada para revisão de código e integração contínua.[2](#)

```
pytest
ruff check src/ tests/
```

Validação local	Resultado
Testes automatizados	9 testes passaram.
Lint com Ruff	Nenhuma violação encontrada.

Observação sobre Pull Requests

A avaliação individual do histórico colaborativo depende de commits e Pull Requests reais feitos no GitHub. O repositório já está preparado para receber esse fluxo, mas cada PR precisa ser criado, revisado e mesclado pelos integrantes reais do grupo. Não é recomendável simular histórico de colaboração, pois o professor avaliará a aba de Pull Requests e os commits vinculados às contas dos alunos.

Referências